MATERIAL DIDÁCTICO · ACCESO A DATOS

TUTORIAL RESUELTO 02 · UNIDAD 02

Transacciones y procedimientos

Commit, rollback, validaciones, CallableStatement y prueba de una operación fallida.

Objetivo y preparación

Aprender a coordinar varias sentencias SQL en una transacción real y llamar a un procedimiento almacenado desde Java. Seguimos con la base MySQL y las tablas creadas en T01.

Paso 1. Arrancar MySQL y comprobar datos

TERMINAL

```
cd Codigo/Infraestructura/mysql
docker compose up -d
docker compose exec mysql mysql -uad_user -p ad_ud2
```

Contraseña: ad_pass. Revisa los saldos antes de ejecutar el programa:

SQL

```
SELECT id, nombre, presupuesto FROM club WHERE id IN (1, 2);
SHOW PROCEDURE STATUS WHERE Db = 'ad_ud2';
EXIT;
```

Si el procedimiento no existe porque el volumen se creó antes de añadirlo al SQL inicial, aplícalo desde un cliente MySQL o utiliza un volumen nuevo únicamente si puedes descartar sus datos.

Paso 2. Abrir el proyecto T02

TERMINAL

```
cd Codigo/T02_Transacciones_Procedimientos
mvn clean compile exec:java
```

El ejemplo **sí cambia** la base en cada ejecución: transfiere 25.00 del club 1 al club 2. No lo ejecutes repetidamente sin comprobar los presupuestos.

Paso 3. Leer el proyecto por archivos

3.1. pom.xml

Ruta: **Codigo/T02_Transacciones_Procedimientos/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>t02-transacciones</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.t02.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

3.2. Abrir una conexión

Ruta: **Codigo/T02_Transacciones_Procedimientos/src/main/java/es/curso/ad/t02/Conexion.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t02;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class Conexion {
    private Conexion() { }
    public static Connection abrir() throws SQLException {
        return DriverManager.getConnection(
                "jdbc:mysql://localhost:3307/ad_ud2?connectionTimeZone=UTC",
                "ad_user", "ad_pass");
    }
}
```

3.3. Transferir y llamar al procedimiento

Ruta: **Codigo/T02_Transacciones_Procedimientos/src/main/java/es/curso/ad/t02/OperacionesClubes.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t02;

import java.math.BigDecimal;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;

public class OperacionesClubes {
    public void transferir(int origenId, int destinoId,
                           BigDecimal importe) throws SQLException {
```

```
        if (origenId == destinoId || importe == null ||
                importe.signum() <= 0) {
            throw new IllegalArgumentException("Transferencia no válida");
        }

        try (Connection conexion = Conexion.abrir()) {
            conexion.setAutoCommit(false);
            try {
                String descuento = "UPDATE club SET presupuesto = presupuesto - ? " +
                        "WHERE id = ? AND presupuesto >= ?";
                try (PreparedStatement sentencia = conexion.prepareStatement(descuento)) {
                    sentencia.setBigDecimal(1, importe);
                    sentencia.setInt(2, origenId);
                    sentencia.setBigDecimal(3, importe);
                    if (sentencia.executeUpdate() != 1) {
                        throw new SQLException("Origen inexistente o saldo insuficiente");
                    }
                }

                String abono = "UPDATE club SET presupuesto = presupuesto + ? WHERE id = ?";
                try (PreparedStatement sentencia = conexion.prepareStatement(abono)) {
                    sentencia.setBigDecimal(1, importe);
                    sentencia.setInt(2, destinoId);
                    if (sentencia.executeUpdate() != 1) {
                        throw new SQLException("El club destino no existe");
                    }
                }

                String registrar = "INSERT INTO movimiento(" +
                        "club_origen_id, club_destino_id, importe) VALUES (?, ?, ?)";
                try (PreparedStatement sentencia = conexion.prepareStatement(registrar)) {
                    sentencia.setInt(1, origenId);
                    sentencia.setInt(2, destinoId);
                    sentencia.setBigDecimal(3, importe);
                    sentencia.executeUpdate();
                }
                conexion.commit();
            } catch (SQLException | RuntimeException e) {
                try {
                    conexion.rollback();
                } catch (SQLException errorRollback) {
                    e.addSuppressed(errorRollback);
                }
                throw e;
            }
        }
    }

    public int contarJugadores(int clubId) throws SQLException {
        try (Connection conexion = Conexion.abrir();
             CallableStatement sentencia = conexion.prepareCall(
                     "{call contar_jugadores(?, ?)}")) {
            sentencia.setInt(1, clubId);
            sentencia.registerOutParameter(2, Types.INTEGER);
            sentencia.execute();
            return sentencia.getInt(2);
        }
    }

    public BigDecimal obtenerPresupuesto(int clubId) throws SQLException {
```

```
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(
                     "SELECT presupuesto FROM club WHERE id = ?")) {
            sentencia.setInt(1, clubId);
            try (ResultSet filas = sentencia.executeQuery()) {
                if (filas.next()) return filas.getBigDecimal(1);
                throw new SQLException("No existe el club " + clubId);
            }
        }
    }
}
```

Observa que transferir usa una sola Connection. Comprueba cada executeUpdate antes de confirmar. El catch realiza rollback y relanza el error para que Main pueda comunicarlo.

3.4. Main y prueba de error intencionada

Ruta: **Codigo/T02_Transacciones_Procedimientos/src/main/java/es/curso/ad/t02/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t02;

import java.math.BigDecimal;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        OperacionesClubes operaciones = new OperacionesClubes();
        try {
            BigDecimal origenAntes = operaciones.obtenerPresupuesto(1);
            BigDecimal destinoAntes = operaciones.obtenerPresupuesto(2);
            System.out.println("Antes: " + origenAntes + " / " + destinoAntes);
            operaciones.transferir(1, 2, new BigDecimal("25.00"));
            System.out.println("Después: " + operaciones.obtenerPresupuesto(1) +
                    " / " + operaciones.obtenerPresupuesto(2));
            System.out.println("Jugadores del club 1: " + operaciones.contarJugadores(1));

            try {
                operaciones.transferir(1, 99999, new BigDecimal("10.00"));
            } catch (SQLException e) {
                System.out.println("Rollback esperado: " + e.getMessage());
            }
            System.out.println("Tras rollback: " + operaciones.obtenerPresupuesto(1));
        } catch (SQLException e) {
            System.err.println("Error SQL: " + e.getMessage());
        }
    }
}
```

El segundo intento utiliza el club destino 99999. Su UPDATE debe afectar a cero filas, lo que provoca rollback y conserva el presupuesto de origen.

Paso 4. Verificar las operaciones en SQL

Entra de nuevo en la consola MySQL y consulta la tabla movimiento. Debe existir una fila por transferencia exitosa, pero no por el intento fallido.

SQL

```
SELECT id, presupuesto FROM club WHERE id IN (1, 2);
SELECT id, club_origen_id, club_destino_id, importe
FROM movimiento ORDER BY id DESC LIMIT 5;
CALL contar_jugadores(1, @total);
SELECT @total AS total_jugadores;
```

Paso 5. Interpretar el código

- setAutoCommit(false) impide confirmar automáticamente cada sentencia por separado.
- commit() confirma los tres cambios; rollback() revierte todos los cambios no confirmados de la transacción.
- CallableStatement registra el parámetro OUT antes de ejecutar y después recupera su valor.
- No usamos double para dinero, sino BigDecimal y DECIMAL.

Paso 6. Casos de prueba guiados

Haz estas pruebas por separado y comprueba el saldo antes y después: una transferencia válida; un destino inexistente; un importe mayor que el saldo; una transferencia entre el mismo club; y un importe cero. Las dos últimas situaciones provocan IllegalArgumentException antes de abrir una conexión.

Resultado imprescindible
Después de una transferencia a un destino inexistente, el origen conserva exactamente el saldo anterior y no aparece un movimiento nuevo. Si el programa descuenta dinero, el código transaccional no está bien implementado.